

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет «Школа информатики, физики и технологий»

[Фамилия Имя Отчество автора]

**Адаптивные топологии мульти-агентных
систем больших языковых моделей с включением
человека в контур управления**

Выпускная квалификационная работа

бакалаврская работа

по направлению подготовки 01.03.02

«Прикладная математика и информатика»

образовательная программа

«Прикладной анализ данных и искусственный интеллект»

Рецензент

Руководитель

[уч. степень, звание]

[уч. степень, звание]

[И.О. Фамилия]

[И.О. Фамилия]

Санкт-Петербург, 2026

Оглавление

Аннотация	4
Abstract	4
Ключевые слова	4
Введение	5
1 Аналитический обзор литературы	10
2 Постановка задачи и методология исследования	18
3 Архитектура программной системы	28
4 Экспериментальное исследование	38
5 Анализ результатов и обсуждение	39
Авторский вклад	39
Заключение	40
Библиографический список	41
А Функциональная схема системы	44
Б Глоссарий	47
В Состав программного репозитория	50
Г Открытый исходный код	52

Аннотация

[TODO] Финал. Заполняется после Главы 5. 150 слов на русском. Краткое описание предметной области, цели, методологии, ключевых результатов и вклада работы.

Abstract

[TODO] Final. Filled after Chapter 5. About 150 words in English.

Ключевые слова

[TODO] Финал. Список из 5–10 ключевых слов или словосочетаний.
Черновик

мульти-агентные системы, большие языковые модели, коммуникационная топология, адаптивные системы, человек в контуре управления, LLM-агенты, оркестрация агентов.

Введение

Актуальность работы. В период с 2023 по 2026 годы применение больших языковых моделей (от англ. Large Language Models, LLM) перешло от одиночных диалоговых сценариев к кооперативной работе нескольких автономных агентов, образующих мульти-агентную систему (от англ. Multi-Agent System, MAS). Крупнейшие открытые фреймворки — AutoGen, ChatDev, MetaGPT, Magentic-One — демонстрируют, что разделение труда между специализированными агентами повышает качество решения сложных задач, недоступных одиночной модели. Однако каждый такой фреймворк жестко фиксирует структуру коммуникации между агентами на этапе проектирования. Систематических данных о том, какая структура является оптимальной для каждого класса задач, в открытой литературе нет. Параллельно растет интерес к интеграции человека в контур управления мульти-агентными системами, но и здесь существующие работы ограничиваются единичными паттернами включения человека без сравнения их эффективности. Сочетание двух этих обстоятельств — интенсивного роста мульти-агентных архитектур и отсутствия систематических знаний о выборе их топологии и позиции человека — определяет научную и прикладную актуальность настоящей работы.

Научная новизна. В настоящей работе впервые проведено прямое сравнение пяти базовых коммуникационных топологий (звезда, цепочка, полносвязная, дебатная, иерархическая) на едином программном стенде, на одной языковой модели и на одной выборке задач из четырех различных классов. Предложена и реализована мета-топология, переключающая активную базовую конфигурацию в зависимости от текущей фазы решения и наблюдаемых сигналов агентов. Введена таксономия из пяти специфичных для адаптивных мульти-агентных систем метрик — N_{switch} , r_{guard} , γ , r_{hurt} , Δ_{LOO} , — позволяющая количественно характеризовать поведение мета-топологии. Впервые сформулирована и экспериментально проверена матрица совместимости пяти ролей человека в контуре управления с пятью базовыми топологиями.

Предмет исследования. Коммуникационная топология мульти-агентной

системы больших языковых моделей, механизм ее динамической адаптации в процессе решения задачи, а также позиция человека в графе обмена сообщениями между агентами.

Объект исследования. Мульти-агентные системы на основе больших языковых моделей, решающие сложные задачи различных классов в кооперации со специализированными агентами и участником-человеком.

Цель работы. Количественная оценка влияния выбора коммуникационной топологии мульти-агентной системы, механизма ее адаптации и позиции человека в контуре управления на эффективность решения задач разных классов с одновременным учетом качества решения, его стоимости и времени выполнения.

Аналитический обзор предлагаемых решений. В публикациях 2022–2026 годов выделяется несколько устойчивых направлений развития мульти-агентных систем больших языковых моделей. Архитектурные фреймворки (AutoGen, ChatDev, MetaGPT) фиксируют топологию на этапе проектирования и не сравнивают альтернативные структуры между собой. Работы об автоматическом проектировании коммуникационных графов (G-Designer, GPTSwarm) подбирают оптимальную структуру под конкретную задачу однократно и не пересматривают ее в процессе решения. Работы по динамической адаптации (DyLAN, AgentVerse) изменяют состав команды агентов, но не саму структуру связей. Вопрос включения человека в контур управления мульти-агентными системами освещен фрагментарно, без сравнительной количественной оценки различных ролей. Подробный анализ всех направлений приведен в главе 1, в частности в сравнительной таблице 1.1 тринадцати методов по семи критериям.

Практическая значимость. Результаты работы предоставляют разработчикам мульти-агентных систем количественные данные о том, какая топология эффективнее для каждого из четырех рассмотренных классов задач, и при каких условиях имеет смысл включать в систему адаптивное переключение топологий. Программная инфраструктура исследования реализована в виде открытого фреймворка ATM (Adaptive Topologies for Multi-agent

systems), полный исходный код которого размещен в репозитории на платформе GitHub. Фреймворк допускает повторное использование и расширение другими исследователями и практиками. Сформулированные в работе правила выбора топологии и роли человека под класс задачи могут применяться при проектировании интеллектуальных ассистентов в программировании, аналитике и принятии решений.

Теоретическая значимость. Введенная в работе таксономия адаптивно-специфичных метрик (N_{switch} , r_{guard} , γ , r_{hurt} , Δ_{LOO}) позволяет количественно характеризовать поведение динамически перестраиваемых мульти-агентных систем и сравнивать их между собой по единой шкале. Формализация мульти-агентной системы как кортежа из шести компонентов с зависящей от фазы коммуникационной структурой расширяет существующую теоретическую базу. Эмпирическая верификация гипотезы о том, что разные фазы решения задачи требуют разных коммуникационных топологий, дает обоснование для дальнейших теоретических работ по теории адаптивных распределенных систем интеллектуальных агентов.

Методы исследования. В работе применен комплекс взаимодополняющих методов. Аналитический обзор литературы с критериями отбора по ключевым словам в открытых базах публикаций. Формализация мульти-агентной системы средствами теории графов и теории конечных автоматов. Программная реализация распределенной системы агентов на языке Python с использованием библиотеки LangGraph для оркестрации. Контролируемый эксперимент по схеме предварительной регистрации гипотез и порогов улучшения. Статистическая обработка результатов методами дисперсионного анализа, парных сравнений с поправкой Бенджамини—Хохберга на множественное сравнение, расчета коэффициента эффекта Коэна и непараметрического бутстрап-оценивания доверительных интервалов. Метод исключения по одному (от англ. leave-one-out, LOO) для построения верхнего потолка адаптации. Имитационное моделирование участника-человека средствами языковой модели с фиксированной для каждой роли подсказкой, обеспечивающее изоляцию вклада роли человека в графе агентов.

Задачи работы. Для достижения сформулированной выше цели в работе решаются следующие задачи.

1. Провести аналитический обзор работ 2022–2026 годов по мульти-агентным системам больших языковых моделей, коммуникационным топологиям и интеграции человека в контур управления, выделить исследовательскую лакуну.
2. Сформулировать формальную модель мульти-агентной системы как кортежа из множества агентов, множества ребер коммуникации, множества ролей агентов и человека, задачи и множества фаз решения.
3. Разработать и реализовать программный фреймворк, в котором пять базовых топологий и адаптивная мета-топология реализованы в единой архитектуре на основе библиотеки LangGraph.
4. Спроектировать систему оценочных метрик, включающую базовые показатели качества, стоимости и времени, а также специфические для адаптивной мета-топологии характеристики переключений, заблокированных решений маршрутизатора, стоимостного вклада маршрутизатора, доли регрессий качества и разрыва до верхнего потолка адаптации.
5. Провести воронку из четырех экспериментов и подтверждающего прогона на четырех классах задач — программирование, математические рассуждения, творческая генерация, анализ данных — с предварительно зафиксированными гипотезами и порогами улучшения.
6. Выполнить статистический анализ результатов и сформулировать ответы на четыре исследовательских вопроса (от англ. Research Questions, RQ), описанных в разделе 2.7.
7. Выработать практические рекомендации по выбору топологии и роли человека в зависимости от класса задачи.

Структура работы. Работа состоит из введения, пяти глав основного содержания, отдельного раздела авторского вклада, заключения, библиографического списка и четырех приложений. Глава 1 содержит аналитический обзор литературы и определяет исследовательскую лакуну. Глава 2 формализу-

ет объект исследования, вводит шесть рассматриваемых топологий, пять ролей человека, систему оценочных метрик, обосновывает выбор корпуса задач и описывает дизайн экспериментальной воронки. Глава 3 описывает архитектуру программной системы, реализующей сформулированный метод. Глава 4 представляет результаты четырех основных экспериментов и подтверждающего прогона на дополнительных семействах моделей. Глава 5 анализирует полученные результаты, дает ответы на четыре исследовательских вопроса и обсуждает угрозы валидности. Раздел авторского вклада явно фиксирует оригинальные элементы работы. Заключение подводит итоги и формулирует направления дальнейших исследований. Приложения содержат функциональную схему системы, глоссарий, краткое описание состава программного репозитория и ссылку на открытый исходный код.

В работе насчитывается 5 глав основного содержания общим объемом около 40 страниц, 5 приложений, не менее 12 таблиц, не менее 8 рисунков, не менее 6 формул и более 30 источников библиографического списка.

1 Аналитический обзор литературы

Настоящая глава представляет аналитический обзор работ, на стыке которых формируется предметная область исследования. Обзор охватывает пять направлений — архитектуру мульти-агентных систем на основе больших языковых моделей, коммуникационные топологии агентов, механизмы их адаптации, безопасность и устойчивость топологий, а также интеграцию человека в контур управления мульти-агентными системами. Глава завершается сравнительной таблицей основных методов и определением исследовательской лакуны, заполнение которой составляет содержательный вклад настоящей работы.

Методология обзора. В рассматриваемую выборку включены работы с 2022 по 2026 годы, отобранные по ключевым словам *multi-agent LLM*, *agent topology*, *LLM debate*, *human-in-the-loop multi-agent* в базах публикаций arXiv, ACL Anthology, ICLR, NeurIPS, ICML. Из первоначального списка из более чем шестидесяти работ оставлены те, которые либо предлагают новую архитектуру взаимодействия агентов, либо систематически анализируют свойства существующих архитектур. Работы, посвященные исключительно обучению одиночных моделей или классическому планированию в дискретных средах, в обзор не включены.

1.1 Мульти-агентные системы больших языковых моделей

Идея использования нескольких автономных программных агентов для совместного решения задач восходит к классическим работам по распределенному искусственному интеллекту. С появлением больших языковых моделей (от англ. Large Language Models, LLM) концепция мульти-агентных систем (от англ. Multi-Agent System, MAS) переживает второе рождение — роль агента теперь играет языковая модель с заданной системной подсказкой и доступным набором инструментов. Обзорная работа [1] систематизирует быстрорастущий корпус исследований и выделяет несколько устойчивых архитектурных образцов.

Первой широко известной системой подобного класса является фреймворк AutoGen [2], в котором несколько агентов обмениваются сообщениями в гибком формате многораундового диалога и могут вызывать внешние инструменты, в том числе подключать человека. AutoGen не фиксирует топологию заранее — структура взаимодействия определяется императивным кодом конкретного сценария. Это позволяет реализовать любую конфигурацию, но лишает фреймворк возможности сравнивать топологии между собой в рамках единой архитектуры.

Иной образец продемонстрирован в системе ChatDev [3], где процесс разработки программного обеспечения моделируется последовательностью ролевых агентов — генерального директора, программиста, тестировщика — работающих по принципу водопадной модели. Метод MetaGPT [4] развивает идею за счет введения стандартных операционных процедур, что позволяет снизить число ошибок согласования между ролями. Эти работы демонстрируют, что разделение труда между специализированными агентами повышает качество решения сложных задач, однако топология взаимодействия в них жестко зашита в архитектуру.

Параллельно развивается направление, в котором отдельный агент итеративно улучшает собственный ответ. Метод Self-Refine [16] реализует цикл генерация — критика — переработка, а метод Reflexion [15] расширяет цикл вербальным подкреплением — модель формирует словесную обратную связь и хранит ее в эпизодической памяти. Несмотря на привлекательную простоту, эти методы по существу остаются одноагентными и не используют разнообразие точек зрения, доступное мульти-агентным конфигурациям.

1.2 Коммуникационные топологии агентов

Под коммуникационной топологией понимается граф, описывающий, какие агенты могут обмениваться сообщениями и в каком порядке. В существующих фреймворках топология выбирается архитектурно и не пересматривается во время выполнения задачи. Тем не менее ряд недавних работ ставит вопрос об оптимальной структуре такого графа. Каноническая таксономия

графовых структур включает пять базовых типов — звезда (центральный координатор и периферийные работники), цепочка (линейный конвейер), полностью связная (каждый агент общается с каждым), дебатная (два оппонента и судья) и иерархическая (двух- или многоуровневое разделение труда). Указанные пять типов и составляют пространство сравнения, исследуемое в настоящей работе.

Работа GPTSwarm [6] рассматривает мульти-агентную систему как обучаемый граф вычислений и оптимизирует его параметры на корпусе задач. Авторы показывают, что оптимизированные графы превосходят случайные конфигурации, но в качестве пространства поиска используют только статические структуры. Метод G-Designer [7] применяет графовые нейронные сети (от англ. Graph Neural Network — GNN) для автоматического подбора топологии под конкретную задачу. Несмотря на адаптацию к типу задачи, выбор топологии производится однократно и не пересматривается в ходе решения.

В работе [8] систематически исследуются закономерности масштабирования мульти-агентных систем при увеличении числа агентов. Авторы обнаруживают, что топологии типа малого мира (small-world) демонстрируют повышенную робастность к ошибкам отдельных агентов, однако сравнение ведется между статическими конфигурациями и не учитывает динамику решения задачи. Метод DyLAN [9] предлагает динамический выбор состава команды агентов на каждом шаге, но варьирует только участников — сама структура связей между ними остается фиксированной. Метод Mixture-of-Agents [11] организует агентов в многослойную ансамблевую конфигурацию с агрегацией ответов, которая также не меняется во время решения.

Промежуточные между однозвенными и мульти-агентными решения представлены работами Tree-of-Thoughts [17] и CAMEL [12]. Первая разворачивает дерево вариантов рассуждения с поиском в ширину, вторая моделирует ролевой диалог двух агентов. В обоих случаях структура взаимодействия определяется заранее и фиксируется на протяжении всей задачи. Ближе к проблеме адаптации структуры подходят работы по дебатам между языковыми моделями [13], где несколько моделей обмениваются аргументами для

повышения качества вывода, однако число оппонентов и протокол обмена в них зафиксированы заранее.

Общая характеристика рассмотренных работ заключается в том, что выбор топологии в них производится либо архитектурно (на этапе проектирования системы), либо заранее (на этапе получения новой задачи). Возможность переключения топологии в процессе решения, в зависимости от фазы и наблюдаемых сигналов, в этих работах не рассматривается.

1.3 Механизмы адаптации топологий

Адаптация мульти-агентных систем во время выполнения исследуется существенно реже. Преобладают два направления адаптации, ни одно из которых не покрывает интересующий нас случай динамического переключения топологии между фазами решения.

Первое направление — адаптация состава команды. Уже упоминавшийся метод DyLAN [9] выбирает на каждом шаге подмножество агентов, наиболее подходящее для текущего контекста. Аналогично работа AgentVerse [10] использует механизм найма агентов, при котором система формирует команду под задачу из заранее заданного пула. В обоих случаях изменяется не структура взаимодействия, а множество участников — сама схема обмена сообщениями остается фиксированной.

Второе направление — оптимизация конфигурации перед запуском задачи. Системы GPTSwarm [6] и G-Designer [7] принимают решение о структуре графа агентов до начала вычислений и не пересматривают его в дальнейшем. Такие методы можно рассматривать как однократную офлайн-адаптацию, которая полезна для классов однородных задач, но недостаточна для задач со сложной фазовой структурой.

Сложные задачи неоднородны по своей внутренней структуре. Фаза планирования может требовать дебатного формата для генерации альтернатив, фаза исполнения — иерархического разделения труда, фаза проверки — независимой рецензии. Отсюда следует естественная гипотеза — статическая топология, оптимальная для усредненной картины, может быть субоптималь-

ной для каждой отдельной фазы. Эта гипотеза мотивирует разработку механизма переключения топологии в процессе выполнения задачи, который и является одним из центральных предметов настоящего исследования.

1.4 Безопасность и устойчивость топологий

Отдельное направление работ посвящено анализу уязвимостей мульти-агентных систем и зависимости этих уязвимостей от топологии. Работа NetSafe [18] систематически исследует устойчивость различных конфигураций к воздействию скомпрометированного агента. Авторы устанавливают, что плотные топологии (полносвязная, звезда) более подвержены каскадному распространению ошибок, тогда как разреженные структуры (цепочка) ограничивают радиус поражения. Работа ТОМА [19] развивает анализ на случай многошаговых атак, в которых злоумышленник может последовательно воздействовать на несколько агентов с учетом их связности.

Эти результаты важны для нашего исследования по двум причинам. Во-первых, они показывают, что выбор топологии не сводится к чистому критерию качества решения — надежность системы к ошибкам отдельных компонентов также является функцией структуры графа. Во-вторых, они мотивируют включение в систему защитных правил переключения топологий, которые предотвращают патологические режимы взаимодействия (в частности, частое осцилляционное переключение между топологиями без существенного прогресса в решении). Реализация таких правил в настоящей работе обсуждается в главе 3.

1.5 Человек в контуре управления

Подключение человека к работе автоматизированных систем (от англ. Human-in-the-Loop, HITL) — давно установленная практика. Обзорная работа [20] систематизирует методы интеграции человека в контур машинного обучения, выделяя такие функции человека как разметка данных, исправление предсказаний, активный отбор примеров. Классическая работа Хорвица [21] ввела принципы смешанной инициативы, при которой автоматизиро-

ванная система и человек поочередно ведут процесс решения задачи в зависимости от уверенности каждой стороны. Эти принципы остаются актуальными для проектирования современных мульти-агентных систем.

Отдельную линию исследований представляют методы обучения языковых моделей с обратной связью от человека. Работа Кристиано и соавторов [22] предложила обучение с подкреплением от человеческих предпочтений, что позднее легло в основу метода InstructGPT [23] и стало стандартной техникой выравнивания моделей. В этих работах человек выступает в роли поставщика сигнала вознаграждения. Применительно к мульти-агентным системам вопрос ставится принципиально иначе — человек участвует не только в обучении, но и в каждом отдельном вызове системы, что требует формализации его роли в графе обмена сообщениями.

Обзор [1] перечисляет несколько типичных паттернов включения человека — координатор, рецензент, судья при споре агентов, — но не предлагает экспериментального сравнения их эффективности. Фреймворк AutoGen [2] допускает подключение человека как агента в диалоге, однако роль человека задается архитектурно и не варьируется в ходе экспериментов. Отдельные работы исследуют человека как судью в задачах оценки качества генерации с использованием шкалы NASA-TLX (от англ. NASA Task Load Index, индекс когнитивной нагрузки NASA) [24], но эта функция выделена в самостоятельную ветвь литературы и не пересекается с проектированием рабочего графа агентов.

Современные оркестраторы мульти-агентных систем, такие как Magentic-One [5], реализуют разделение между ведущим агентом-координатором и подчиненными работниками, при этом человек явно подключен как опциональный участник в роли постановщика задачи. Однако сравнения эффективности разных ролей человека в этой работе также не проводится.

Систематических исследований того, какая роль человека и в какой фазе наиболее эффективна для разных типов задач, в проанализированной выборке работ обнаружить не удалось. Эта лакуна составляет второй центральный предмет настоящего исследования.

1.6 Сравнительная аналитическая таблица методов

Рассмотренные методы естественно сопоставить по нескольким критериям, релевантным для исследовательских вопросов работы. Соответствующее сравнение приведено в таблице 1.1. Знак «+» означает наличие свойства, «−» — его отсутствие, «±» — частичную реализацию.

Таблица 1.1. Сравнение существующих методов по ключевым критериям

Метод	Год	MAS	Сравн. топ.	Адапт. состава	Адапт. топ. runtime	HITL
ReAct [14]	2022	−	−	−	−	−
CAMEL [12]	2023	+	−	−	−	−
AutoGen [2]	2023	+	−	−	−	+
Tree-of-Thoughts [17]	2023	−	−	−	−	−
Reflexion [15]	2023	−	−	−	−	−
DyLAN [9]	2023	+	−	+	−	−
AgentVerse [10]	2023	+	−	+	−	−
ChatDev [3]	2024	+	−	−	−	−
MetaGPT [4]	2024	+	−	−	−	−
GPTSwarm [6]	2024	+	±	−	−	−
G-Designer [7]	2024	+	+	−	−	−
Self-Refine [16]	2024	−	−	−	−	−
MoA [11]	2024	+	−	−	−	−
Настоящая работа	2026	+	+	−	+	+

Обозначения колонок — **MAS** мульти-агентная архитектура, **Сравн. топ.** систематическое сравнение нескольких топологий на едином стенде, **Адапт. состава** динамическое изменение участников, **Адапт. топ. runtime** переключение структуры графа в процессе решения задачи, **HITL** интеграция человека в контур управления как полноценного участника графа.

1.7 Что отсутствует в существующих работах

Из таблицы 1.1 видно, что три столбца — систематическое сравнение топологий на едином стенде, адаптация топологии во время решения задачи, явная роль человека в структуре графа — в существующих работах представлены лишь эпизодически и в проанализированной выборке не встречаются одновременно. Метод G-Designer [7] систематически сравнивает топологии, но не адаптирует их во время выполнения задачи. AutoGen [2] позволяет подключать человека, но не формализует его роль и не сравнивает разные конфигурации. DyLAN [9] адаптирует состав, но не структуру связей. Ни один из методов не объединяет в одном эксперименте сравнение нескольких топологий, механизм их переключения во время решения и многомерное варьи-

рование роли человека.

Эта совокупность отсутствующих свойств и составляет исследовательскую лакуну, заполнение которой является целью настоящей работы. В частности, настоящая работа предлагает — во-первых, единый программный стенд, на котором сравниваются пять различных топологий на едином корпусе из четырех классов задач; во-вторых, мета-топологию, переключающую активную базовую топологию в зависимости от фазы решения; в-третьих, формализацию пяти ролей человека и экспериментальную оценку их влияния на каждую топологию по полному факторному плану.

1.8 Выводы по главе

Проведенный аналитический обзор позволяет сделать несколько содержательных выводов. Мульти-агентные системы больших языковых моделей являются активно развивающейся областью с устоявшимся набором архитектурных образцов, однако коммуникационная топология в них обычно фиксируется на этапе проектирования и не пересматривается во время выполнения задачи. Существующие механизмы адаптации ограничены либо составом команды, либо однократным офлайн-выбором структуры графа, что недостаточно для задач со сложной фазовой структурой. Безопасность мульти-агентных систем существенно зависит от топологии, что подтверждает необходимость защитных правил при ее динамическом изменении. Систематического исследования роли человека в графе агентов в открытой литературе не обнаружено.

На пересечении трех направлений — сравнение топологий, их динамическая адаптация, формализованная роль человека — лежит лакуна, заполнение которой ставится в качестве научной цели настоящей работы. Эта цель формализуется в виде четырех исследовательских вопросов в разделе 2.

2 Постановка задачи и методология исследования

Настоящая глава формализует объект исследования, фиксирует исследовательские вопросы и описывает методологию их эмпирической проверки. Сначала вводится математическая модель мульти-агентной системы, затем определяются шесть рассматриваемых коммуникационных топологий и пять ролей человека в контуре управления. Далее формулируется система оценочных метрик, обосновывается выбор корпуса задач и моделей, и описывается воронка из четырех экспериментов, последовательно отвечающих на поставленные исследовательские вопросы.

2.1 Формальная модель мульти-агентной системы

Мульти-агентная система рассматривается как кортеж

$$\mathcal{M} = \langle V, E, R, \mathcal{T}, F, h \rangle, \quad (2.1)$$

где $V = \{v_1, \dots, v_n\}$ — конечное множество агентов, каждый из которых реализован как языковая модель с фиксированной системной подсказкой и набором инструментов; $E \subseteq V \times V$ — множество допустимых направленных ребер коммуникации, образующее коммуникационный граф; $R: V \rightarrow \mathcal{R}$ — отображение агентов в множество ролей $\mathcal{R}$ (планировщик, исследователь, исполнитель, критик, дебатер, координатор); $\mathcal{T}$ — текущая задача с целевой функцией оценки качества решения; F — конечное множество фаз решения задачи, представленных автоматом $\{planning, execution, verification, done\}$; h — (опционально) участник-человек с одной из пяти заданных ролей.

В классических работах граф E определяется архитектурно и не меняется в ходе выполнения задачи. В настоящем исследовании вводится расширение модели — структура E становится функцией текущей фазы $f \in F$ и

наблюдаемых сигналов σ агентов:

$$E = E(f, \sigma, t), \quad (2.2)$$

где t — индекс итерации. Возможность такого переключения и составляет суть исследуемой адаптивной мета-топологии.

Множество ролей фиксировано и состоит из шести значений — $\mathcal{R} = \{planner, researcher, executor, critic, debater, coordinator\}$. Множество ролей человека фиксировано и содержит пять значений, формализованных в разделе 2.3. Фазы F упорядочены отношением $planning \prec execution \prec verification \prec done$ с монотонной семантикой переходов. Сигналы σ агентов представляют собой вектор булевых индикаторов фиксированной длины, в который входят флаги завершения этапа, признаки тупикового состояния, счетчики отказов критика и другие наблюдаемые признаки прогресса.

Задача оптимизации мульти-агентной системы формулируется как многокритериальная и состоит в нахождении конфигурации $\langle E, R, h \rangle$, доминирующей конкурирующие конфигурации одновременно по трем осям — качеству решения $q(\mathcal{M})$, суммарной стоимости вызовов $c(\mathcal{M})$ и времени выполнения $\tau(\mathcal{M})$. Поскольку чистый Парето-оптимум обычно не единственен, в работе используется скаляризация с фиксированными порогами улучшения. Конфигурация $\mathcal{M}_1$ считается выигрышной относительно $\mathcal{M}_0$, если выполняется одно из условий

$$\frac{q(\mathcal{M}_1) - q(\mathcal{M}_0)}{q(\mathcal{M}_0)} \geq 0,05 \quad \text{или} \quad \frac{c(\mathcal{M}_0) - c(\mathcal{M}_1)}{c(\mathcal{M}_0)} \geq 0,15 \quad \text{при} \quad q(\mathcal{M}_1) \geq q(\mathcal{M}_0). \quad (2.3)$$

Пороги 5% для качества и 15% для стоимости зафиксированы до проведения основных прогонов и обоснованы в разделе 2.4.

2.2 Шесть рассматриваемых топологий

Пространство сравнения составляют шесть топологий. Пять из них являются статичными и реализуют пять базовых классов коммуникационных

структур, перечисленных в обзоре литературы. Шестая является мета-топологией, переключающей активную статическую топологию в зависимости от фазы.

Star — центральный координатор v_c последовательно обращается к специализированным работникам $v_1, \dots, v_k$ и агрегирует их ответы. Граф ребер $E_{\text{star}} = \{(v_c, v_i), (v_i, v_c) \mid i = 1, \dots, k\}$.

Chain — линейный конвейер из трех агентов $v_p \rightarrow v_e \rightarrow v_c$ (планировщик, исполнитель, критик) с условным циклом доработки. Граф направленный ациклический с обратной связью от критика к исполнителю.

Mesh — полносвязная топология с общей шиной сообщений. Граф $E_{\text{mesh}} = (V \times V) \setminus \{(v, v)\}$, обмен организован по принципу циклической очереди, решение принимается голосованием.

Debate — пара оппонентов v_+ и v_- генерирует параллельные альтернативные решения, после чего судья v_j выбирает победителя или синтезирует ответ. Граф двудольный с агрегатором.

Hierarchical — двухуровневая иерархия из координатора верхнего уровня v_c^{top} и двух подкоманд $T_a, T_b \subset V$, каждая из которых сама по себе является графом. Подкоманды работают параллельно, координатор синтезирует итог.

Adaptive — мета-топология, активирующая на каждой итерации одну из пяти базовых конфигураций. Решение о выборе принимается маршрутизатором согласно формуле (2.2). Псевдокод алгоритма приведен в таблице 2.1, общая концептуальная схема взаимодействия блоков системы изображена на рисунке 3.1 главы 3, архитектура реализации описана в разделе 3.3.

2.3 Пять ролей человека в контуре управления

Пять ролей человека формализованы по аналогии с типовыми позициями в человеческих организациях. **Координатор** (ρ_C) — управляет планом и делегированием, активен в иерархических и полносвязных топологиях. **Рецензент** (ρ_R) — проверяет промежуточные результаты и инициирует доработку, естественно вписывается в Star и Chain. **Судья** (ρ_J) — выносит финальный вердикт при наличии альтернатив, релевантен для Debate. **Равноправный участник** (ρ_P) — вносит альтернативные мнения наравне с аген-

Таблица 2.1. Алгоритм работы адаптивной мета-топологии

Шаг	Действие
1	Инициализировать фазу $f \leftarrow planning$, начальную топологию $K \leftarrow K_0$, состояние $s \leftarrow s_0$ и журнал $\mathcal{J} \leftarrow \emptyset$.
2	Пока $f \neq done$ и $c(s) < B$ выполнять шаги 3–10.
3	Применить маршрутизатор фаз $f' \leftarrow PhaseRouter(s, f)$.
4	Применить маршрутизатор топологий $K' \leftarrow TopologyRouter(s, K, f')$.
5	Если $Guard(s, K, K') = block$, то $K' \leftarrow K$ и записать $guard_override$ в $\mathcal{J}$.
6	Если $K' \neq K$, применить ворота перехода состояния $s \leftarrow TransitionGate(s, K, K')$ и записать переход в $\mathcal{J}$.
7	Исполнить подграф выбранной топологии $s \leftarrow Subgraph_{K'}(s)$.
8	Обновить $K \leftarrow K'$ и $f \leftarrow f'$.
9	Записать журнал итерации в $\mathcal{J}$.
10	Перейти к шагу 2.
11	Вернуть результат $y(s)$ и журнал $\mathcal{J}$.

тами, вписывается в Mesh. **Наблюдатель** (ρ_M) — следит за процессом и вмешивается только в критических случаях (превышение бюджета, зависание, циклы), применим в любой топологии.

Не все пары (топология, роль) являются осмысленными. Пара (Debate, Coordinator) исключается, поскольку координатор в дебатной топологии эквивалентен судье. Пары (Chain, Peer) и (Hierarchical, Peer) исключаются, поскольку равноправный участник ломает линейную или иерархическую структуру. После исключений пространство содержит 22 осмысленные комбинации топология $\times$ роль.

2.4 Система оценочных метрик

Метрики разделены на три группы. Первая группа — базовые метрики качества и эффективности — применима ко всем экспериментам и ко всем топологиям.

- q — агрегированный показатель качества решения. Зависит от типа задачи и принимает значения в $[0, 1]$. Для задач программирования — доля прошедших юнит-тестов, для математических задач — точное совпадение числового ответа, для творческой генерации — среднее метрики ROUGE-L (от англ. Recall-Oriented Understudy for Gisting Evaluation, Longest common subsequence variant) и доли покрытия заданных кон-

цептов, для анализа данных — точное совпадение числового ответа.

- c — суммарная стоимость вызовов языковых моделей в долларах США за один прогон.
- τ — время выполнения прогона по настенным часам (wall-clock) в секундах.
- r_{fail} — доля прогонов, завершившихся со статусом, отличным от «выполнено».
- N_{iter} — число итераций основного цикла.

Вторая группа — метрики, специфичные для адаптивной мета-топологии.

- N_{switch} — число фактических переключений активной топологии за один прогон.
- r_{guard} — доля решений маршрутизатора, заблокированных защитными правилами.
- γ — доля бюджета, потраченная маршрутизатором на собственные вызовы языковой модели; критичная метрика, отвечающая на вопрос, не превышает ли стоимость адаптации получаемый от нее выигрыш.
- r_{hurt} — доля задач, на которых адаптивная конфигурация дает качество хуже лучшей статической конфигурации. Является ключевым индикатором безопасности механизма адаптации.
- Δ_{LOO} — разрыв качества между фактическим маршрутизатором и оракулом, построенным по процедуре leave-one-out (для каждой задачи оптимальная топология вычисляется на остальных задачах того же класса). Эта метрика служит честным верхним потолком адаптации.

Третья группа — метрики, относящиеся к взаимодействию с человеком.

- N_{int} — число обращений системы к человеку за один прогон.
- L_{cog} — прокси-метрика когнитивной нагрузки, агрегирующая число обращений, длину предоставляемого контекста и среднюю задержку ответа симулятора человека. В экспериментах с реальными участниками заменяется на стандартную шкалу NASA-TLX.

Адаптивная конфигурация считается выигрышной относительно наилучшей статической, если она дает прирост качества не менее 5% либо сниже-

ние стоимости не менее 15% при сохранении качества. Пороги фиксируются предварительно, до проведения основных прогонов, что соответствует практике предварительной регистрации гипотез в эмпирических исследованиях.

2.5 Корпус задач и обоснование его выбора

Эксперименты проводятся на стратифицированной выборке из четырех открытых корпусов, покрывающих четыре качественно различных класса задач — программирование, математические рассуждения, творческая генерация и анализ данных. Сравнение корпуса с альтернативами приведено в таблице 2.2.

Таблица 2.2. Сравнение рассмотренных корпусов задач

Корпус	Класс	Размер	Метрика	Решение по включению
HumanEval [25]	Программирование	164	pass@1	Включен; стандарт сравнения.
HumanEval+ [26]	Программирование	164	pass@1+	Не включен; превышает покрытие основной выборки.
LiveCodeBench [27]	Программирование	500+	pass@1	Резерв для проверки на загрязнение.
GSM8K [28]	Математ. рассуждения	8500	exact match	Включен; стандарт оценки.
MATH [29]	Математ. рассуждения	12500	exact match	Не включен; сложность выше возможностей рабочих моделей.
MMLU-Pro [30]	Множеств. рассужд.	12000	accuracy	Не включен; формат ответа не подходит для дебатной топологии.
CommonGen [31]	Творческая генерация	35000	ROUGE-L + cov	Включен; покрывает open-ended генерацию с ограничениями.
InfiAgent-DABench [32]	Анализ данных	257	numeric exact	Включен; единственный корпус с замкнутым ответом по аналитике данных.
HellaSwag [33]	Здравый смысл	10000	accuracy	Не включен; качество современных моделей близко к потолку.

Выбор обусловлен совокупностью требований. Каждый класс задач представлен ровно одним корпусом, чтобы избежать смешения эффектов корпуса и класса. Каждый корпус допускает автоматическую объективную оценку, исключая субъективность оценщика (юнит-тесты, точное совпадение числа, метрики покрытия концептов). Размер задач в каждом корпусе поз-

воляет уложиться в один прогон рабочих моделей. От рассматриваемых для каждого класса альтернатив выбранные корпуса отличаются распространенностью в литературе и зрелостью методики оценки, что обеспечивает сопоставимость результатов с предшествующими работами. Из каждого корпуса производится стратифицированная выборка по 15 задач, итого 60 задач в одном базовом запуске.

2.6 Языковые модели и инфраструктура

Распределение моделей по ролям спроектировано таким образом, чтобы изолировать переменную «модель» от переменных «топология» и «роль человека». Все рабочие агенты (планировщик, исследователь, исполнитель, критик, дебатер), а также модели маршрутизаторов и симулятора человека используют одну и ту же модель Cerebras gpt-oss-120b, обеспечивающую низкую стоимость вызовов и высокую скорость генерации (порядка трех тысяч токенов в секунду на инфраструктуре Cerebras WSE). Иерархия «работник слабее критика» реализуется не разной моделью, а различиями в системных подсказках и в параметре строгости рассуждения. Это решение принципиально для чистоты эксперимента, поскольку фиксирует одну и ту же модельную мощность для всех сравниваемых конфигураций.

Судья оценки качества решений выделен в отдельную линию моделей — OpenAI gpt-4.1-mini с тремя независимыми вызовами по схеме самосогласованности и нулевой температурой выборки. Использование модели из другой семейства весов является ключевым методологическим выбором — оно исключает риск того, что система оценивает собственные ответы и формирует петлю положительной обратной связи. Подтверждающие прогоны (см. ниже) проводятся на двух кросс-семейных моделях — Cerebras zai-glm-4.7 и OpenAI gpt-4o, — что позволяет разделить эффекты адаптивной топологии от эффектов конкретного семейства весов.

Выбор основных моделей опирался на четыре критерия. Во-первых, доступность отказоустойчивого программного интерфейса с поддержкой вызова инструментов и структурированного вывода — требование, обеспечиваю-

щее воспроизводимое исполнение многошаговых циклов агентов. Во-вторых, низкая стоимость единичного вызова, позволяющая выполнить полную сетку из нескольких тысяч прогонов в рамках бюджета бакалаврской работы. В-третьих, высокая пропускная способность, исключающая инфраструктурные ограничения как причину различий между топологиями. В-четвертых, доступность из Российской Федерации без юридических и финансовых препятствий. Под эти критерии попадают Cerebras gpt-oss-120b (основная инфраструктура), OpenAI gpt-4.1-mini (судья оценки) и две подтверждающие модели — Cerebras zai-glm-4.7 как представитель другого семейства весов на той же инфраструктуре и OpenAI gpt-4o как frontier-модель другого провайдера. Подробный сравнительный анализ альтернативных моделей и поставщиков приведен в техническом отчете проекта.

Инфраструктура экспериментов описана в главе 3 и включает оркестрацию на LangGraph, хранение метаданных в PostgreSQL версии 16, объемных журналов — в формате Apache Parquet. Параллелизм выполнения настроен на двенадцать процессов одновременно с асинхронной обработкой запросов внутри каждого процесса.

2.7 Дизайн экспериментов — воронка

Прямой эксперимент по всему декартову произведению переменных (шесть топологий, пять ролей человека, три режима маршрутизации, четыре класса задач) дал бы более десяти тысяч ячеек и потребовал бы непропорционального бюджета. Вместо этого исследование организовано как последовательность из четырех экспериментов, каждый из которых сужает пространство конфигураций по результатам предыдущего. Структура воронки приведена в таблице 2.3.

Исследовательские вопросы формулируются следующим образом.

RQ1. Различаются ли пять статических топологий по количественным показателям качества, стоимости и времени для разных классов задач?

RQ2. Дает ли адаптивная мета-топология значимый прирост качества или экономию стоимости по сравнению с лучшей статической конфигураци-

Таблица 2.3. Структура экспериментальной воронки

Эксп.	Иссл. вопрос	Прогоны	Цель
E1	RQ1	900	Pareto-фронт пяти статических топологий на четырех классах задач.
E2	RQ3	2160	Влияние пяти ролей человека на каждую топологию.
E3	RQ2	720	Адаптивная мета-топология против лучшей статической конфигурации в трех режимах маршрутизации.
E4	RQ4	540	Адаптивная роль человека в сочетании с адаптивной топологией.
Conf.	—	1100	Кросс-семейная валидация выводов на двух подтверждающих моделях.

ей?

RQ3. Какая из пяти ролей человека наиболее эффективна для каждой топологии и для каждого класса задач?

RQ4. Превосходит ли совместная адаптация топологии и роли человека по фазам решения статичные комбинации?

Статистическая обработка включает дисперсионный анализ с эффектом топологии и эффектом роли в качестве факторов, парные сравнения средних с поправкой Бенджамини—Хохберга на множественное сравнение, расчет коэффициента эффекта Коэна для оценки практической значимости различий и доверительные интервалы по методу непараметрического бутстрапа с тысячью повторений.

2.8 Разграничение заимствованных и авторских элементов

Поскольку методология настоящей работы сочетает несколько существующих элементов и предлагает несколько новых, ниже явно фиксируется граница между ними. К заимствованному относятся пять статических топологий (звезда, цепочка, полносвязная, дебатная, иерархическая) как известные паттерны организации мульти-агентного взаимодействия, четыре открытых корпуса задач, языковые модели и стандартный статистический инструментарий (дисперсионный анализ, метод бутстрапа, коэффициент эффекта Коэна). К авторским относятся следующие элементы. Во-первых, адаптивная мета-топология вместе с алгоритмом ее работы, сформулированная в разделе

ле 2.2 и формализованная уравнением (2.2). Во-вторых, формализация пяти ролей человека и соответствующая матрица совместимости топологий с ролями. В-третьих, набор адаптивно- специфичных метрик (N_{switch} , r_{guard} , γ , r_{hurt} , Δ_{LOO}), формализованных в разделе 2.4. В-четвертых, вороночная структура эмпирического исследования с предварительной фиксацией порогов из формулы (2.3) и кросс-семейным подтверждением на двух дополнительных моделях.

2.9 Выводы по главе

В главе формализована мульти-агентная система как кортеж из шести компонентов с возможностью динамического переключения структуры коммуникационного графа в зависимости от фазы решения. Определены шесть рассматриваемых топологий, пять ролей человека в контуре управления и три группы оценочных метрик, включая специфичные для адаптивной мета-топологии характеристики — число переключений, долю заблокированных решений маршрутизатора, стоимостный вклад самого маршрутизатора, долю задач с регрессией качества и разрыв до оракула. Обоснован выбор корпуса задач из четырех открытых бенчмарков и линии языковых моделей с кросс-семейным судьей. Сформулированы четыре исследовательских вопроса и описана воронка из четырех основных экспериментов с подтверждающим прогоном на дополнительных семействах моделей. Граница между заимствованными и предложенными в работе элементами зафиксирована в разделе 2.8 и развернута в самостоятельном разделе авторского вклада (5). Изложенный план непосредственно реализуется в инфраструктуре главы 3 и эмпирически проверяется в главе 4.

3 Архитектура программной системы

Программная инфраструктура, обеспечивающая выполнение всех описанных в главе 2 экспериментов, реализована в виде самостоятельного исследовательского фреймворка с условным именем АТМ (от англ. Adaptive Topologies for Multi-agent systems — адаптивные топологии для мульти-агентных систем). Фреймворк написан на языке Python версии не ниже 3.11 и распространяется как пакет `atm`. Настоящая глава описывает архитектурное устройство системы с акцентом на те решения, которые имеют непосредственное отношение к научным гипотезам работы, — адаптивному переключению топологий и интеграции человека в контур управления. Полный исходный код размещен в открытом репозитории, ссылка приведена в приложении Г.

Общая функциональная схема системы изображена на рисунке 3.1. На вход подается описание задачи из выбранного бенчмарка и конфигурация эксперимента в формате YAML (от англ. YAML Ain't Markup Language — человекочитаемый формат сериализации). На выход поступают журналы взаимодействия агентов, агрегированные метрики качества и стоимости, а также структурированные записи о фазах решения, переключениях топологий и взаимодействиях с человеком. Внутри между входом и выходом последовательно работают пять архитектурных блоков — оркестратор эксперимента, менеджер фаз и маршрутизаторы, активная топология с агентами, шлюз взаимодействия с человеком и подсистема наблюдаемости.

3.1 Обзор архитектуры и слои

Кодовая база организована в виде тринадцати функционально изолированных модулей-слоев, каждый из которых отвечает за один аспект системы. Перечень слоев и их назначений приведен в таблице 3.1. Такая декомпозиция позволяет независимо тестировать компоненты, заменять реализации (например, провайдера языковой модели или шлюз HITL — от англ. Human-in-the-Loop — человек в контуре управления) и добавлять новые топологии без изменения существующего кода.

Рисунок 3.1. Функциональная схема предлагаемой системы.

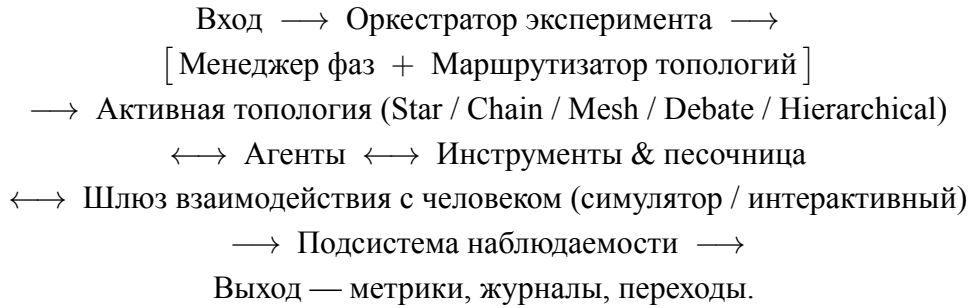


Рисунок 3.1. Функциональная схема предлагаемой мульти-агентной системы. Прямоугольниками выделены архитектурные блоки фреймворка, двунаправленные стрелки изображают многократные обмены сообщениями в пределах одной итерации.

Таблица 3.1. Функциональные слои фреймворка atm

Слой	Назначение
core	Базовые типы и состояние графа — сообщения, фазы, роли, контейнеры состояния AgentState, SharedState, GraphState.
llm	Унифицированная оболочка над провайдерами больших языковых моделей, подсчет стоимости, бюджетные ограничители, детерминированный FakeLLM для тестов.
tools	Реестр инструментов и Docker-песочница для исполнения генерируемого кода.
agents	Реализация пяти ролей агентов и базовый класс Agent с управляющим циклом вызова инструментов.
topology	Шесть реализаций топологий поверх LangGraph и общий протокол Topology.
phases	Конечный автомат фаз, маршрутизатор топологий и защитные правила переключения.
human	Протокол HumanGateway, симулированный и интерактивный шлюзы, маршрутизатор ролей человека.
storage	Объектно-реляционное отображение, асинхронные сессии PostgreSQL, запись в формат Apache Parquet.
observability	Перехват событий LangGraph, сериализация и отправка их в хранилище.
tasks	Реализация и загрузки бенчмарков HumanEval, GSM8K, CommonGen, InfiAgent-DABench.
evaluation	Судьи на основе языковой модели, оракулы достоверности, агрегатор шкалы NASA-TLX (от англ. NASA Task Load Index — индекс когнитивной нагрузки NASA).
experiment	Запуск одиночных и грид-экспериментов, схемы конфигов, командная строка.
analysis	Загрузки результатов, агрегаторы метрик, построение графиков.

Архитектурный стиль фреймворка опирается на несколько последовательно применяемых паттернов. Во-первых, все расширяемые точки описаны как протоколы языка Python (Topology, HumanGateway, PhaseRouter, TopologyRouter, RoleRouter, Tool) с проверкой соответствия в момент выполнения через декоратор `@runtime_checkable`. Это позволяет подключать сторонние реализации без явного наследования. Во-вторых, для сообщений и переходов используется паттерн редуктора. Состояние графа объединяется при параллельном выполнении агентов через явно зарегистрированные функции слияния, гарантирующие идемпотентность и дедупликацию по идентификатору сообщения. В-третьих, повторяющиеся сущности — топологии, инструменты, оракулы — регистрируются в реестрах с декораторной семантикой.

Технологический стек системы выбран из соображений воспроизводимости и масштабируемости. Оркестрация графов агентов выполнена на библиотеке LangGraph, использующей чекпойнтер для возобновления прерванных запусков. Метаданные экспериментов хранятся в PostgreSQL версии 16 через асинхронные сессии SQLAlchemy 2.x, объемные журналы взаимодействия — в файлах Apache Parquet. Конфигурации экспериментов описаны на YAML и валидируются композитными моделями библиотеки Pydantic поверх OmegaConf. Линтер ruff и статический анализатор типов mypy применяются в строгом режиме.

Часть архитектурных решений заимствована из существующих работ и фреймворков, часть является авторской разработкой. Точное разграничение приведено в таблице 3.2.

3.2 Слой агентов и оболочка над языковой моделью

Базовый класс Agent в модуле `atm.agents` реализует типовой управляющий цикл агента — считывание входящих сообщений, формирование подсказки с учетом скрэтчпада, вызов языковой модели, обработку запрошенных инструментов и публикацию ответа. На основе этого класса определены пять ролей — Planner, Researcher, Executor, Critic, Debater — и дополнительная роль Coordinator, активируемая в иерархических и адаптивных то-

Таблица 3.2. Разделение архитектурных компонентов на заимствованные и авторские

Компонент	Источник	Авторский вклад
Оркестрация графов	LangGraph (внешний)	Адаптация под мета-граф
Пять статичных топологий	Прототипы из обзора литературы	Унифицированная реализация по-верх об-щего прото-токо-ла
Адаптивная мета-топология	Своя разработка	Полностью
Маршрутизаторы фаз	Своя разработка	Полностью, три ре-жима (пра-вило-вый, на ос-нове язы-ковой мо-дели, ора-кул)
Защитные правила переключения	Своя разработка	Полностью, че-тыре типа огра-ничи-телей
Шлюз HITL	Своя разработка	Полностью, три роле-вых марш-рути-затора и поли-тика тайм-аутов

пологиях. Каждая роль определяется YAML-конфигом, содержащим системную подсказку, допустимый набор инструментов, размер скользящего окна сообщений, температуру выборки и параметры управления контекстом.

Скрэтчпад агента (внутренний рабочий контекст) организован по политике С. История сообщений поддерживается в виде скользящего окна фиксированного размера, а при его переполнении старые сообщения сжимаются вспомогательной языковой моделью в краткое резюме. Компромисс между полнотой истории и стоимостью вызовов выбран по результатам пилотного эксперимента с двумя альтернативными политиками (полная история и плоский буфер без сжатия), описанного в разделе 4.

Оболочка `LLMWrapper` в модуле `atm.llm` унифицирует работу с различными провайдерами — OpenAI, Anthropic, Cerebras и локальным FakeLLM. Класс инкапсулирует подсчет токенов и стоимости вызова на основе таблицы цен `Pricing`, реализует экспоненциальный повтор при временных сбоях провайдера и фиксирует снимок версии модели в первом успешном ответе. Бюджетные ограничители `BudgetGuard` применяются на трех уровнях — одиночный вызов, полный запуск эксперимента, грид-эксперимент в целом, — и при превышении лимита прерывают выполнение с явной диагностикой. Реализация FakeLLM поддерживает три режима — сценарный, эхо и воспроизведение по записи, — что обеспечивает детерминированное повторение экспериментов и работоспособность модульных тестов без обращения к внешним сервисам.

3.3 Реализация топологий поверх LangGraph

Каждая из шести топологий реализована как самостоятельный класс, удовлетворяющий протоколу `Topology` с двумя обязательными полями — именем (`name`) и фабричным методом (`build`), возвращающим скомпилированный направленный граф состояния `LangGraph`. Граф связывает узлы — агентов и управляющие функции — ребрами трех типов — безусловными, условными и параллельными через примитив `Send` библиотеки `LangGraph`. Все построенные графы единообразно подключаются к чекпойнтеру и к пе-

рехватчику событий слоя `atm.observability`, что обеспечивает однородность журнала вне зависимости от выбранной топологии.

Пять статичных топологий различаются принципом организации коммуникации. В топологии *Star* центральный координатор последовательно вызывает специализированных работников и агрегирует их ответы. Топология *Chain* реализует линейный конвейер с обратными связями — планировщик, исполнитель, критик — и условным циклом доработки. *Mesh* использует общую шину сообщений с диспетчером по принципу циклической очереди и голосованием за финальное решение. *Debate* запускает двух оппонентов параллельно (`debater_pro` и `debater_contra`) с последующим выбором победителя судьей. *Hierarchical* организует двухуровневую иерархию из координатора верхнего уровня и двух подкоманд, каждая из которых представляет собой самостоятельный скомпилированный подграф.

Шестая топология *Adaptive* устроена как мета-граф второго уровня. Каждый ее тик начинается с обращения к маршрутизатору фаз и маршрутизатору топологий, после чего выбранная базовая топология вызывается как подграф. По завершении подграфа ворота переходов (`TransitionGate`) применяют таблицу правил передачи состояния и записывают объект `TopologyTransition` в журнал. Скомпилированные подграфы кэшируются по имени, что исключает повторную компиляцию `LangGraph` при многократных переключениях. Решения маршрутизаторов хранятся в замыкании, а не в общем состоянии, чтобы избежать их перезаписи во время выполнения вложенного подграфа.

3.4 Менеджер фаз и маршрутизатор топологий

Решение задачи в системе моделируется конечным автоматом (от англ. `Finite State Machine` — `FSM`) из четырех состояний — планирование, исполнение, проверка и завершение, — переходы между которыми инициируются сигналами агентов и проверками максимального числа итераций. Слой `atm.phases` содержит две реализации маршрутизатора фаз. Детерминированная реализация `RuleBasedPhaseRouter` использует таблицу сигналов и монотонную проверку допустимости перехода — фаза не может сместиться

назад. Маршрутизатор на основе языковой модели (LLMPhaseRouter) обращается к модели с шаблоном подсказки, парсит результат как структурированный объект формата JSON (от англ. JavaScript Object Notation) и автоматически откатывается на правило при любой ошибке валидации или генерации.

Параллельно с фазами работает маршрутизатор топологий, отвечающий за решение о переключении базовой топологии внутри адаптивной конфигурации. Реализованы три режима. Правильный RuleBasedTopologyRouter использует таблицу (фаза $\times$ сигналы) $\rightarrow$ топология, где сигнал stuck в фазе исполнения активирует *Mesh*, а сигнал rejected_count со значением не ниже трех — *Debate*. Конкретные пороги срабатывания подобраны эмпирически в пилотном эксперименте и обоснованы в разделе 4. LLMTopologyRouter принимает решение на основе подсказки с описанием текущего состояния и допустимых топологий, с аналогичным правилowym откатом. OracleTopologyRouter читает заранее построенную таблицу решений из файла, что служит честным верхним потолком адаптации.

Защитные правила переключения, реализованные в виде набора замыканий, предотвращают режим частого осцилляционного переключения, при котором система непрерывно меняет активную топологию без существенного прогресса. Применяются четыре ограничения — минимальное число итераций в одной топологии перед сменой (min_dwell), интервал между сменами (cooldown), максимальное число переключений за фазу и за весь запуск. Если решение маршрутизатора заблокировано хотя бы одним из ограничений, оно фиксируется в журнале со специальной отметкой guard_override, что позволяет в дальнейшем оценить долю заблокированных решений как метрику стабильности.

3.5 Шлюз взаимодействия с человеком

Интеграция человека в контур управления абстрагирована единым протоколом HumanGateway, единственным методом которого является асинхронный запрос ответа от человека по описанию контекста (запрос содержит идентификатор запуска и запроса, текущую роль человека, недавнюю историю

сообщений и допустимый набор действий). Идемпотентность гарантируется парой идентификаторов (`run_id`, `request_id`). Повторный вызов с тем же идентификатором возвращает ранее полученный ответ без обращения к человеку, что критично для возобновления прерванных запусков после сбоев.

Реализованы два шлюза. `LLMSimulatedGateway` использует языковую модель в роли симулятора человека с системной подсказкой, зависящей от выбранной роли — координатор, рецензент, судья, равноправный участник или наблюдатель. `CLIGateway` ориентирован на интерактивное взаимодействие через стандартный поток ввода-вывода и применяется в экспериментах с реальными участниками. Маршрутизатор ролей человека `RoleRouter` выбирает активную роль динамически — правилочная таблица или решение языковой модели — либо фиксированно (`FixedRoleRouter`). Политика обработки тайм-аутов отвечает за сценарии, в которых человек не успевает ответить в отведенное время. Доступны три варианта — откат к языковой модели, пропуск шага, повтор, — что делает поведение системы предсказуемым в любых условиях.

3.6 Хранение, командная строка и воспроизводимость

Метаданные экспериментов хранятся в семи таблицах PostgreSQL — `experiments`, `runs`, `phases`, `topology_transitions`, `human_interactions`, `budget_events`, `llm_calls`. Схема включает индексы по типичным аналитическим запросам (по эксперименту, по топологии, по статусу) и составной индекс по паре (эксперимент, статус) для быстрой выборки невыполненных запусков. Объемные журналы взаимодействия агентов записываются в файлы Apache Parquet через асинхронный писатель и партиционируются по идентификатору запуска. Миграции схемы базы данных управляются библиотекой Alembic, текущая ревизия фиксирует поддержку воспроизведения запусков.

Командная строка `atm`, реализованная на библиотеке Typer, объединяет семь команд для запуска и сопровождения экспериментов, описание которых приведено в таблице 3.3.

Таблица 3.3. Команды интерфейса командной строки фреймворка

Команда	Назначение
atm run	Запуск одиночного эксперимента из YAML-конфига.
atm grid	Параллельный грид-эксперимент через ProcessPoolExecutor.
atm estimate	Предварительная оценка стоимости запуска по таблице цен и историческим данным.
atm status	Агрегированный статус эксперимента — число выполненных, неудачных и активных ячеек, суммарная стоимость.
atm resume	Возобновление прерванного запуска из последнего чекпойнта LangGraph.
atm replay	Детерминированное воспроизведение запуска через FakeLLM в режиме воспроизведения по записи.
atm reconcile	Поиск и пометка запусков-зомби (запись о статусе «выполняется», но процесс уже завершен).

Воспроизводимость обеспечивается набором инвариантов, фиксируемых в момент старта каждого запуска. Функция `seed_all` иницирует все известные источники случайности (Python, NumPy, провайдеры языковых моделей, генератор идентификаторов). Значение `git_sha` извлекается вызовом `git rev-parse HEAD` и записывается в таблицу экспериментов. Снимок версий используемых моделей формируется в первом успешном ответе от каждого провайдера и сохраняется в поле `model_version_snapshot` запуска. Дайджест образа Docker-песочницы фиксируется при ее инициализации. Совокупность этих полей делает возможным сравнение результатов между запусками и анализ артефактов депрекации моделей со стороны провайдеров.

3.7 Выводы по главе

Программная инфраструктура работы реализована как тринадцатислойный фреймворк на языке Python с акцентом на расширяемость через протоколы, воспроизводимость через фиксацию инвариантов состояния и наблюдаемость через единый перехват событий LangGraph. В рамках этой инфраструктуры представлены шесть топологий, два режима маршрутизации фаз, три режима маршрутизации топологий и два шлюза взаимодействия с человеком — то есть все компоненты, необходимые для выполнения экспериментов главы 4. Подробное разграничение между заимствованными и авторски-

ми компонентами приведено в таблице 3.2, сформулировано отдельно как авторский вклад в разделе 5.

Объемные характеристики реализации. Кодовая база основного пакета `atm` содержит около 21 300 строк кода на языке Python, распределенных по 112 модулям. Конфигурация экспериментов задается через 43 файла формата YAML, из которых 24 описывают непосредственные параметры запусков серии E1–E4. Приведенные числовые характеристики получены прямым измерением исходной кодовой базы на момент сдачи работы; служебные файлы — модульные и интеграционные проверки, скрипты сборки, документация проекта — в подсчет не включены.

4 Экспериментальное исследование

[**TODO**] Этап 8. Блок до получения данных E1–E4.

5 Анализ результатов и обсуждение

[TODO] Этап 8. Блок до завершения Главы 4.

Авторский вклад

[TODO] Этап 8 заключительный. Три пункта вклада. 1. Программная инфраструктура мульти-агентного фреймворка — пять статичных топологий, адаптивная мета-топология, шлюз HITL, три режима маршрутизации, leave-one-out оракул. 2. Таксономия метрик для адаптивных мульти-агентных систем. 3. Эмпирические результаты на четырех классах задач.

Заключение

[**TODO**] Этап 8 заключительный. 1. Главный результат работы. 2. Краткие ответы на RQ1–RQ4. 3. Ограничения работы. 4. Направления дальнейших исследований — минимум 10 строк связным текстом (валидация на реальных пользователях, кросс-семейная валидация на frontier-моделях, масштабирование на большой корпус задач, применение к производственным сценариям).

Библиографический список

- [1] A survey on large language model based autonomous agents / L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, J.-R. Wen // *Frontiers of Computer Science*. – 2024. – Vol. 18, no. 6. – P. 186345.
- [2] AutoGen: Enabling next-gen LLM applications via multi-agent conversation [Электронный ресурс] / Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, C. Wang // *arXiv preprint 2308.08155*. – 2023. – URL: <https://arxiv.org/abs/2308.08155> (дата обращения 17.05.2026).
- [3] ChatDev: Communicative agents for software development / C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, Z. Liu // *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*. – 2024. – P. 15174–15186.
- [4] MetaGPT: Meta programming for a multi-agent collaborative framework / S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, Y. Wu // *Proceedings of the International Conference on Learning Representations (ICLR 2024)*. – 2024.
- [5] Magentic-One: A generalist multi-agent system for solving complex tasks [Электронный ресурс] / A. Fourney, G. Bansal, H. Mozannar, C. Tan, E. Salinas, E. Nieldtner, E. Horvitz // *arXiv preprint 2411.04468*. – 2024. – URL: <https://arxiv.org/abs/2411.04468> (дата обращения 17.05.2026).
- [6] GPTSwarm: Language agents as optimizable graphs / M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, J. Schmidhuber // *Proceedings of the International Conference on Machine Learning (ICML 2024)*. – 2024.
- [7] G-Designer: Architecting multi-agent communication topologies via graph neural networks [Электронный ресурс] / Y. Zhang, K. Xu, Y. Li, Z. Liu, D. Shen // *arXiv preprint 2410.11782*. – 2024. – URL: <https://arxiv.org/abs/2410.11782> (дата обращения 17.05.2026).
- [8] Scaling large-language-model-based multi-agent collaboration [Электронный ресурс] / C. Qian, Z. Ye, W. Liu, N. Chen, Z. Liu // *arXiv preprint 2406.07155*. – 2024. – URL: <https://arxiv.org/abs/2406.07155> (дата обращения 17.05.2026).
- [9] Dynamic LLM-agent network: an LLM-agent collaboration framework with agent team optimization [Электронный ресурс] / Z. Liu, Y. Zhang, P. Li, Y. Liu, D. Yang // *arXiv preprint 2310.02170*. – 2023. – URL: <https://arxiv.org/abs/2310.02170> (дата обращения 17.05.2026).
- [10] AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors [Электронный ресурс] / W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu // *arXiv preprint 2308.10848*. – 2023. – URL: <https://arxiv.org/abs/2308.10848> (дата обращения 17.05.2026).
- [11] Mixture-of-Agents enhances large language model capabilities / J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, J. Zou // *Proceedings of the Conference on Language Modeling (COLM 2024)*. – 2024.
- [12] CAMEL: Communicative agents for "mind" exploration of large language model society / G. Li, H. Hammoud, H. Itani, D. Khizbullin, B. Ghanem // *Advances in Neural Information Processing Systems*. – 2023. – Vol. 36.
- [13] Improving factuality and reasoning in language models through multiagent debate [Электронный ресурс] / Y. Du, S. Li, A. Torralba, J. Tenenbaum, I. Mordatch // *arXiv preprint 2305.14325*. – 2023. – URL: <https://arxiv.org/abs/2305.14325> (дата обращения 17.05.2026).
- [14] ReAct: Synergizing reasoning and acting in language models / S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao // *Proceedings of the International Conference on Learning Representations (ICLR 2023)*. – 2023.

- [15] Reflexion: Language agents with verbal reinforcement learning / N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao // *Advances in Neural Information Processing Systems*. – 2023. – Vol. 36.
- [16] Self-Refine: Iterative refinement with self-feedback / A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri // *Advances in Neural Information Processing Systems*. – 2023. – Vol. 36.
- [17] Tree of Thoughts: Deliberate problem solving with large language models / S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, K. Narasimhan // *Advances in Neural Information Processing Systems*. – 2023. – Vol. 36.
- [18] NetSafe: Exploring the topological safety of multi-agent networks against adversarial attacks [Электронный ресурс] / M. Yu, S. Wang, G. Zhang, H. Li // *arXiv preprint 2410.15686*. – 2024. – URL: <https://arxiv.org/abs/2410.15686> (дата обращения 17.05.2026).
- [19] Tipping the dominos: Topology-aware multi-hop attacks on LLM-based multi-agent systems [Электронный ресурс] / R. Liang, J. Ye, Z. Chen, Q. Zhu // *arXiv preprint 2412.04129*. – 2024. – URL: <https://arxiv.org/abs/2412.04129> (дата обращения 17.05.2026).
- [20] Human-in-the-loop machine learning: a state of the art / E. Mosqueira-Rey, E. Hernández-Pereira, D. Alonso-Ríos, J. Bobes-Bascarán, Á. Fernández-Leal // *Artificial Intelligence Review*. – 2023. – Vol. 56, no. 4. – P. 3005–3054.
- [21] Horvitz E. Principles of Mixed-Initiative User Interfaces / E. Horvitz // *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '99)*. – New York: ACM, 1999. – P. 159–166.
- [22] Deep reinforcement learning from human preferences / P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, D. Amodei // *Advances in Neural Information Processing Systems*. – 2017. – Vol. 30.
- [23] Training language models to follow instructions with human feedback / L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal // *Advances in Neural Information Processing Systems*. – 2022. – Vol. 35. – P. 27730–27744.
- [24] Hart S. G. Development of NASA-TLX (Task Load Index): results of empirical and theoretical research / S. G. Hart, L. E. Staveland // *Human Mental Workload* / ed. by P. A. Hancock, N. Meshkati. – Amsterdam: North-Holland, 1988. – P. 139–183.
- [25] Evaluating large language models trained on code [Электронный ресурс] / M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda // *arXiv preprint 2107.03374*. – 2021. – URL: <https://arxiv.org/abs/2107.03374> (дата обращения 17.05.2026).
- [26] Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation / J. Liu, C. S. Xia, Y. Wang, L. Zhang // *Advances in Neural Information Processing Systems*. – 2023. – Vol. 36.
- [27] LiveCodeBench: Holistic and contamination-free evaluation of large language models for code [Электронный ресурс] / N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama // *arXiv preprint 2403.07974*. – 2024. – URL: <https://arxiv.org/abs/2403.07974> (дата обращения 17.05.2026).
- [28] Training verifiers to solve math word problems [Электронный ресурс] / K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek // *arXiv preprint 2110.14168*. – 2021. – URL: <https://arxiv.org/abs/2110.14168> (дата обращения 17.05.2026).
- [29] Measuring mathematical problem solving with the MATH dataset / D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, J. Steinhardt // *Advances in Neural Information Processing Systems*. – 2021. – Vol. 34.

- [30] MMLU-Pro: A more robust and challenging multi-task language understanding benchmark [Электронный ресурс] / Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj // arXiv preprint 2406.01574. – 2024. – URL: <https://arxiv.org/abs/2406.01574> (дата обращения 17.05.2026).
- [31] CommonGen: A constrained text generation challenge for generative commonsense reasoning / B. Y. Lin, W. Zhou, M. Shen, P. Zhou, C. Bhagavatula, Y. Choi, X. Ren // Findings of the Association for Computational Linguistics (EMNLP 2020). – 2020. – P. 1823–1840.
- [32] InfiAgent-DABench: Evaluating agents on data analysis tasks [Электронный ресурс] / X. Hu, Z. Zhao, S. Wei, Z. Chai, Q. Ma, G. Wang, X. Wang, J. Su, J. Xu, M. Zhu, Y. Cheng, J. Yuan // arXiv preprint 2401.05507. – 2024. – URL: <https://arxiv.org/abs/2401.05507> (дата обращения 17.05.2026).
- [33] HellaSwag: Can a machine really finish your sentence? / R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, Y. Choi // Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL 2019). – 2019. – P. 4791–4800.
- [34] Lin C.-Y. ROUGE: A package for automatic evaluation of summaries / C.-Y. Lin // Text Summarization Branches Out: Proceedings of the ACL-04 Workshop. – Barcelona, 2004. – P. 74–81.
- [35] Cohen J. Statistical power analysis for the behavioral sciences / J. Cohen. – 2nd ed. – Hillsdale, NJ: Lawrence Erlbaum Associates, 1988. – 567 p.
- [36] Benjamini Y. Controlling the false discovery rate: a practical and powerful approach to multiple testing / Y. Benjamini, Y. Hochberg // Journal of the Royal Statistical Society. Series B. – 1995. – Vol. 57, no. 1. – P. 289–300.
- [37] Efron B. Bootstrap methods: another look at the jackknife / B. Efron // The Annals of Statistics. – 1979. – Vol. 7, no. 1. – P. 1–26.
- [38] AgentBench: Evaluating LLMs as agents [Электронный ресурс] / X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding // arXiv preprint 2308.03688. – 2023. – URL: <https://arxiv.org/abs/2308.03688> (дата обращения 17.05.2026).
- [39] RouteLLM: Learning to route LLMs with preference data [Электронный ресурс] / I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, I. Stoica // arXiv preprint 2406.18665. – 2024. – URL: <https://arxiv.org/abs/2406.18665> (дата обращения 17.05.2026).
- [40] Chen L. FrugalGPT: How to use large language models while reducing cost and improving performance / L. Chen, M. Zaharia, J. Zou // Transactions on Machine Learning Research. – 2024.
- [41] LangGraph: A framework for building stateful, multi-actor applications with LLMs [Электронный ресурс] / LangChain AI. – 2024. – URL: <https://langchain-ai.github.io/langgraph/> (дата обращения 17.05.2026).
- [42] Chase H. LangChain: Building applications with LLMs through composability [Электронный ресурс] / H. Chase. – 2023. – URL: <https://github.com/langchain-ai/langchain> (дата обращения 17.05.2026).
- [43] Pydantic: Data validation using Python type hints [Электронный ресурс] / S. Colvin et al. – 2024. – URL: <https://docs.pydantic.dev/> (дата обращения 17.05.2026).
- [44] LangSmith: Observability and evaluation for LLM applications [Электронный ресурс] / LangChain AI. – 2024. – URL: <https://docs.smith.langchain.com/> (дата обращения 17.05.2026).

Приложение А Функциональная схема системы

В настоящем приложении приведена развернутая функциональная схема программной системы, описанной в главе 3. Схема изображает движение управления и данных между основными архитектурными блоками от поступления входных данных эксперимента до получения результирующих метрик и журналов. На рисунке А.1 сплошными стрелками показано движение управления, штриховыми — обмен сообщениями между агентами, точечными — потоки записи в подсистему наблюдаемости и хранения.

Пояснение к схеме

Поток выполнения начинается с поступления конфигурации эксперимента в формате YAML и описания задачи T из выбранного корпуса. Оркестратор эксперимента (`atm.experiment`) инициализирует общее состояние графа и передает управление слою адаптации. Маршрутизатор фаз принимает решение о текущей фазе решения, передает результат маршрутизатору топологий, который выбирает активную коммуникационную топологию. В работе одновременно действует ровно одна из пяти базовых топологий, а адаптивный режим реализуется через последовательное переключение между ними по решению маршрутизатора.

Активная топология определяет порядок активации агентов и протокол обмена сообщениями между ними. Любая из шести ролей агентов (планировщик, исследователь, исполнитель, критик, дебатер, координатор) может быть подключена к любой топологии в соответствии с правилами раздела 2.2. Агенты обращаются за внешними службами — инструментами и Docker-песочницей для исполнения кода, оболочкой над языковыми моделями для генерации текста, шлюзом взаимодействия с человеком.

Все события системы (вызовы языковых моделей, выставление сигналов, переходы между фазами, переключения топологий, обращения к человеку, срабатывания защитных правил маршрутизатора) перехватываются подсистемой наблюдаемости и направляются в два хранилища — реляционную

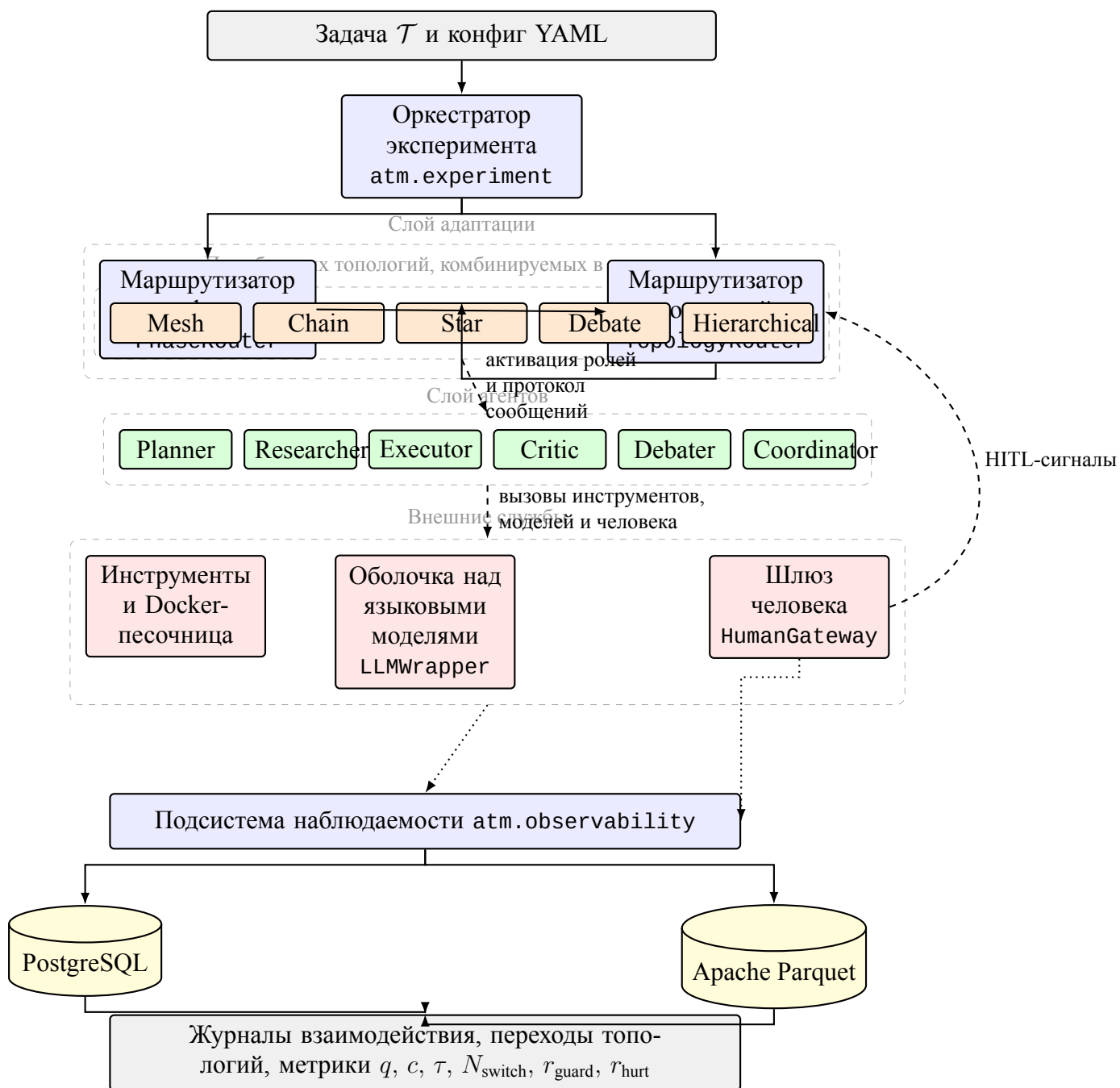


Рисунок А.1. Функциональная схема программной системы. Сплошные стрелки — передача управления; штриховые — обмен сообщениями между блоками и обратный канал от человека к маршрутизатору топологий; точечные — потоки записи событий в подсистему наблюдаемости. Цветовое кодирование — серый цвет соответствует точкам входа и выхода, синий — управляющим компонентам, оранжевый — активным коммуникационным топологиям, зеленый — слою агентов, красный — внешним службам исполнения, желтый — подсистеме хранения.

базу для метаданных и колоночные файлы Apache Parquet для объемных журналов. На финальной стадии результаты прогона становятся доступными для постобработки слою анализа `atm.analysis`.

Обратный канал от шлюза человека к маршрутизатору топологий обозначает поступление управляющих сигналов от участника-человека к адаптивному слою. Через этот канал человек, действующий в роли координатора или наблюдателя, может форсировать переключение топологии или досрочно завершить фазу проверки. Точечная связь от шлюза к подсистеме наблюдаемости отвечает за регистрацию взаимодействий с человеком, на основе которой вычисляются метрики r_{guard} и r_{hurt} .

Приложение Б Глоссарий

В приложении приведены определения ключевых терминов работы. Термины сгруппированы по тематическим блокам.

Мульти-агентные системы и языковые модели

Большая языковая модель (от англ. Large Language Model, LLM) — глубокая нейронная сеть, обученная на текстовом корпусе для предсказания следующего токена. **Мульти-агентная система** (от англ. Multi-Agent System, MAS) — программная система из нескольких автономных агентов, взаимодействующих между собой и с внешней средой. **Агент** — программная сущность с фиксированной ролью, системной подсказкой, набором инструментов и собственным состоянием. **Системная подсказка** — постоянная инструкция, задающая агенту роль, ограничения и стиль ответов; не меняется в пределах одного запуска. **Скрэтчпад** (от англ. scratchpad) — локальный приватный контекст агента со скользящим окном истории сообщений и автоматическим сжатием при переполнении. **Рабочий агент** (worker) — агент, непосредственно участвующий в решении задачи; противопоставляется управляющим компонентам. **Судья оценки качества** (LLM-as-judge) — внешняя языковая модель из другого семейства весов, оценивающая качество финального ответа системы с самосогласованностью. **Симулятор человека** — языковая модель в роли участника с системной подсказкой, зависящей от заданной роли. **Состояние графа** (graph state) — общий контейнер контекстов агентов, текущей фазы, активной топологии и журнала переходов. **Бюджет прогона** — трехуровневый ограничитель стоимости вызовов языковых моделей на отдельный вызов, на полный прогон и на грид-эксперимент.

Коммуникационные топологии

Коммуникационная топология — направленный граф допустимых маршрутов обмена сообщениями между агентами и порядка их активации. **Топология Star** — центральный координатор последовательно вызывает специализированных работников и агрегирует их ответы. **Топология Chain** — линейный конвейер из трех агентов с обратными связями для доработки. **Топология Mesh** — полносвязная конфигурация с общей шиной сообщений и циклическим обменом. **Топология Debate** — два оппонента параллельно генерируют альтернативные решения, судья выбирает победителя. **Топология Hierarchical** — двухуровневая иерархия из координатора верхнего уровня и двух подкоманд. **Адаптивная метатопология** — конфигурация с выбором активной базовой топологии на каждой итерации в зависимости от фазы и сигналов агентов; авторская разработка.

Фазы и маршрутизаторы

Фаза решения задачи — состояние конечного автомата из четырех значений (планирование, исполнение, проверка, завершение). **Конечный автомат (FSM)** — математическая модель с конечным множеством состояний и переходов. **Сигнал агента** — булев индикатор готовности к следующей фазе, тупикового состояния, отказа критика. **Маршрутизатор фаз (PhaseRouter)** — компонент принятия решения о переходе процесса в следующую фазу; правилковый, на основе языковой модели или оракул. **Маршрутизатор топологий (TopologyRouter)** — компонент смены активной базовой топологии внутри адаптивной мета-топологии. **Защитные правила переключения (SwitchGuards)** — ограничения против частого осцилляционного переключения (минимальное число итераций в топологии, интервал между сменами, лимиты за фазу и за запуск). **Ворота передачи состояния (TransitionGate)** — компонент применения правил передачи состояния при смене активной топологии и регистрации в журнале.

Человек в контуре управления

Человек в контуре управления (HITL) — парадигма, при которой автоматизированная система обращается к человеку за решением, оценкой или подтверждением. **Шлюз взаимодействия с человеком (HumanGateway)** — программный протокол с идемпотентностью повторных вызовов по паре идентификаторов запуска и запроса. **Роль координатора** — человек управляет планом и делегированием задач агентам. **Роль рецензента** — человек проверяет промежуточные результаты и инициирует доработку. **Роль судьи** — человек выносит финальный вердикт при наличии альтернативных решений. **Роль равноправного участника (peer)** — человек вносит собственные сообщения в общую шину наравне с агентами. **Роль наблюдателя (monitor)** — человек пассивно следит за процессом и вмешивается только в критических случаях. **Маршрутизатор ролей (RoleRouter)** — компонент динамического выбора активной роли человека по фазе и наблюдаемому состоянию. **Шкала NASA-TLX** — стандартный инструмент субъективной оценки когнитивной нагрузки по шести шкалам.

Метрики

Качество решения (q) — агрегированный показатель в $[0, 1]$; конкретная формула зависит от класса задач. **Полная стоимость прогона (c)** — сумма стоимостей вызовов языковых моделей в долларах США за один прогон. **Время прогона (τ)** — полное время выполнения по настенным часам. **Доля отказов (r_{fail})** — доля прогонов со статусом, отличным от завершённого. **Число переключений топологии (N_{switch})** — фактическое число смен активной базовой топологии за прогон адаптивной мета-топологии. **Доля заблокированных реше-**

ний маршрутизатора (r_{guard}) — доля решений, заблокированных защитными правилами. **Стоимостный вклад маршрутизатора** (γ) — доля бюджета, потраченная на собственные вызовы маршрутизатора. **Доля регрессий качества** (r_{hurt}) — доля задач с качеством хуже лучшей статической конфигурации; ключевой индикатор безопасности адаптации. **Разрыв до оракула** (Δ_{LOO}) — разность между качеством оракула и фактического маршрутизатора; верхняя граница достижимого качества. **Прокси-метрика когнитивной нагрузки** (L_{cog}) — агрегированный показатель из числа обращений, объема контекста и средней задержки ответа симулятора.

Инфраструктура

LangGraph — библиотека Python для построения и исполнения направленных графов состояния с асинхронными вызовами и чекпойнтером. **Чекпойнтер** — механизм сохранения промежуточного состояния графа для возобновления прерванных запусков. **Apache Parquet** — колоночный формат хранения структурированных данных для журналов взаимодействия. **Грид-эксперимент** — запуск серии прогонов по декартову произведению значений нескольких параметров с параллельным исполнением. **Прогон** (run) — один полный цикл выполнения мульти-агентной системы; элементарная единица измерения.

Эксперимент и статистика

Корпус задач — размеченное множество тестовых задач одного класса с автоматической процедурой оценки. **Воронка экспериментов** — последовательность экспериментов с сужением пространства конфигураций по результатам предыдущих. **Подтверждающий прогон** — дополнительный эксперимент на отличной языковой модели для проверки независимости выводов от семейства весов. **Дисперсионный анализ** (ANOVA) — метод проверки гипотезы о равенстве средних в нескольких группах. **Поправка Бенджамини—Хохберга** — процедура контроля доли ложных открытий при множественном сравнении. **Коэффициент эффекта Козна** (d) — стандартизированная мера величины различия двух средних. **Метод бутстрапа** — непараметрический метод оценивания распределения статистики через ресэмплирование выборки; применяется для доверительных интервалов с уровнем 95 % и тысячей повторений. **Доверительный интервал** — интервал, накрывающий истинное значение параметра распределения с заданной вероятностью. **Метод исключения по одному** (leave-one-out, LOO) — процедура построения верхнего потолка адаптации с вычислением оптимальной конфигурации на остальных задачах того же класса. **Конфигурация системы** — набор значений всех переменных эксперимента; различают статические (с фиксированной топологией) и адаптивные (с переключением).

Приложение В Состав программного репозитория

В приложении приведено краткое описание состава программного репозитория проекта. Состав фиксирован по состоянию идентификатора фиксации, указанного в приложении Г.

Корневая структура

src/atm/ — основной пакет фреймворка, тринадцать функциональных слоев. **conf/** — декларативная конфигурация экспериментов и компонентов в формате YAML. **scripts/** — самостоятельные скрипты обслуживания экспериментов и сборки производных артефактов. **notebooks/** — шаблон Jupyter-ноутбука для аналитической постобработки результатов прогонов. **pyproject.toml** — декларация пакета, зависимостей и точки входа **atm**. **README.md** — точка входа в документацию проекта и инструкция по сборке среды.

Слои пакета **src/atm/**

atm.core — контейнеры состояния и базовые типы (**AgentState**, **SharedState**, **GraphState**, **Phase**, **HumanRole**). **atm.llm** — оболочка над провайдерами языковых моделей, ценообразование, бюджетные ограничители, детерминированный **FakeLLM**, фабрика **build_llm**. **atm.tools** — реестр инструментов и обертка над **Docker**-песочницей. **atm.agents** — базовый класс **Agent** и пять специализированных ролей плюс координатор. **atm.topology** — шесть реализаций топологий (**star.py**, **chain.py**, **mesh.py**, **debate.py**, **hierarchical.py**, **adaptive.py**) поверх общего протокола **Topology**. **atm.phases** — конечный автомат фаз, маршрутизатор топологий, защитные правила переключения, ворота передачи состояния. **atm.human** — протокол **HumanGateway**, реализации шлюзов, маршрутизатор ролей, политика тайм-аутов. **atm.storage** — объектно-реляционное отображение, модели таблиц **PostgreSQL**, асинхронные сессии, писатель **Parquet**, обертка над чекпойнтером **LangGraph**, миграции **alembic/**. **atm.observability** — перехватчик событий **LangGraph**, сериализация, обертки над логгером. **atm.tasks** — загрузчики и оценщики четырех корпусов (**HumanEval**, **GSM8K**, **CommonGen**, **InfiAgent-DABench**). **atm.evaluation** — судья оценки качества, оракулы достоверности, агрегатор шкалы **NASA-TLX**. **atm.experiment** — схемы конфигов на **Pydantic**, загрузчик **OmegaConf** с расширением по сетке, командная строка **atm** на **Type**. **atm.analysis** — загрузчики результатов, агрегаторы метрик, генератор графиков, построитель оракула.

Конфигурационные файлы **conf/**

conf/experiments/ — полные конфигурации экспериментов **E1–E4**, подтверждающих прогонов и проверок работоспособности. **conf/agents/** — системные подсказки шести ролей

агентов. **conf/topology/** — параметры по умолчанию для шести топологий. **conf/human/** — параметры шлюза HITL и системные подсказки симулятора. **conf/model/** — параметры языковых моделей. **conf/task/, conf/tasks/** — загрузчики корпусов и стратифицированные выборки. **conf/oracle/** — конфигурации построения оракула. **conf/evaluation/** — параметры судьи и агрегаторов. **conf/sandbox/** — параметры Docker-песочницы и профиль системных вызовов.

Скрипты и ноутбуки

scripts/gen_analysis_notebook.py — генератор аналитического ноутбука из шаблона.

scripts/derive_seccomp.py — сборка профиля системных вызовов для песочницы. **notebooks/anal** — шаблонный ноутбук из шести секций для постобработки и визуализации результатов прогона.

Приложение Г Открытый исходный код

В соответствии с пунктом 5.5 Правил подготовки и защиты выпускных квалификационных работ образовательной программы «Прикладной анализ данных и искусственный интеллект» полная программная реализация настоящего исследования размещена в открытом доступе. В качестве платформы размещения выбран сервис GitHub. Выбор обусловлен наличием встроенного механизма версионирования с уникальными идентификаторами фиксаций (коммитов), что обеспечивает однозначное соответствие текста работы конкретному состоянию исходного кода, а также поддержкой стандартных открытых лицензий и широкой распространенностью платформы в академической и инженерной практике.

Адрес репозитория [URL репозитория на платформе GitHub — заполняется автором перед сдачей. Пример формата <https://github.com/<user>/adaptive-topologies-mas>]

Идентификатор фиксации [Хеш фиксации репозитория на момент сдачи работы — заполняется автором.]

Лицензия Лицензия публикации указана в файле LICENSE репозитория.

Сборка среды Полная инструкция по установке зависимостей, развертыванию базы данных PostgreSQL и проверке корректности сборки приведена в файле README.md.

Воспроизведение результатов Конфигурации экспериментов главы 4 размещены в директории conf/experiments/ и снабжены примерами запуска через командную строку atm.

Согласие на публикацию работы

В соответствии с пунктом 5.6.3 Правил подготовки и защиты выпускных квалификационных работ согласие автора на публикацию настоящей выпускной квалификационной работы на портале Национального исследовательского университета «Высшая школа экономики» оформлено в виде QR-кода системы «LMS-Антиплагиат», прикрепленного к итоговому отчету о

проверке работы. Указанный отчет передан в составе пакета документов в Государственную экзаменационную комиссию.